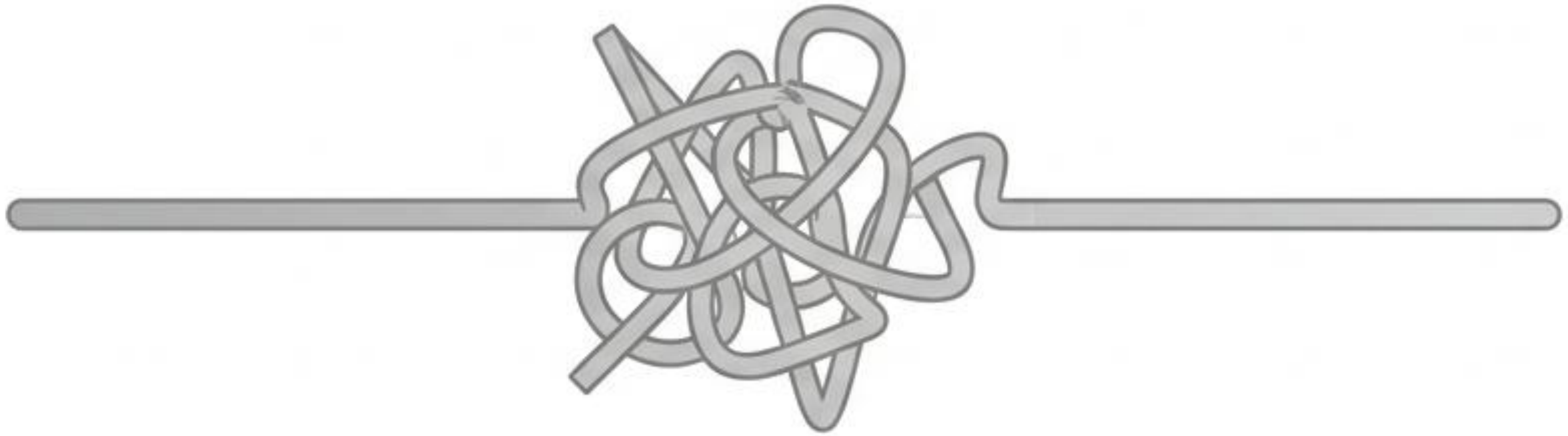
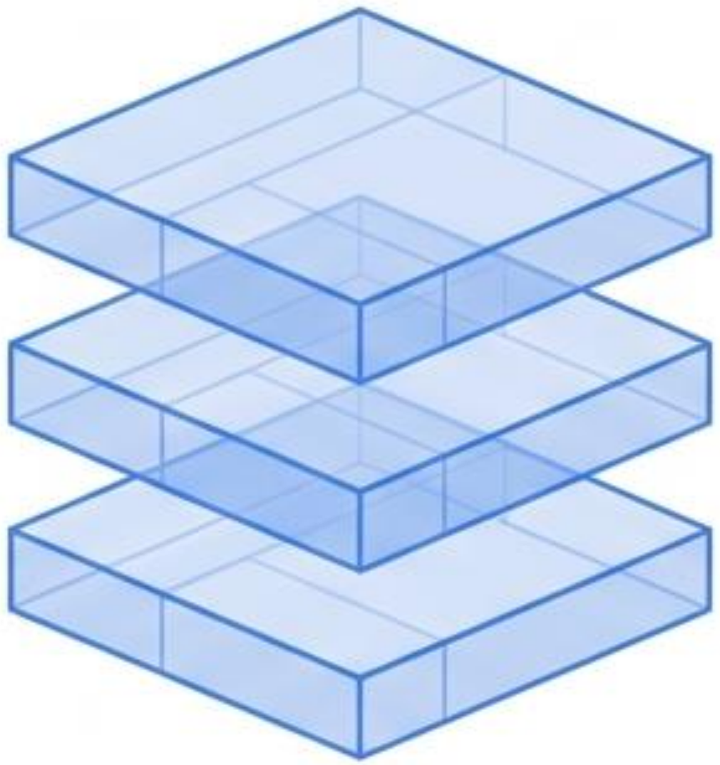
resumazing

ResuMazing: The Journey of an Intelligent CV Optimizer

High-value career advice is trapped inside unsearchable video formats





The Bottleneck

Video content is linear. Users cannot instantly query a one-hour YouTube video for specific resume advice without watching it entirely.

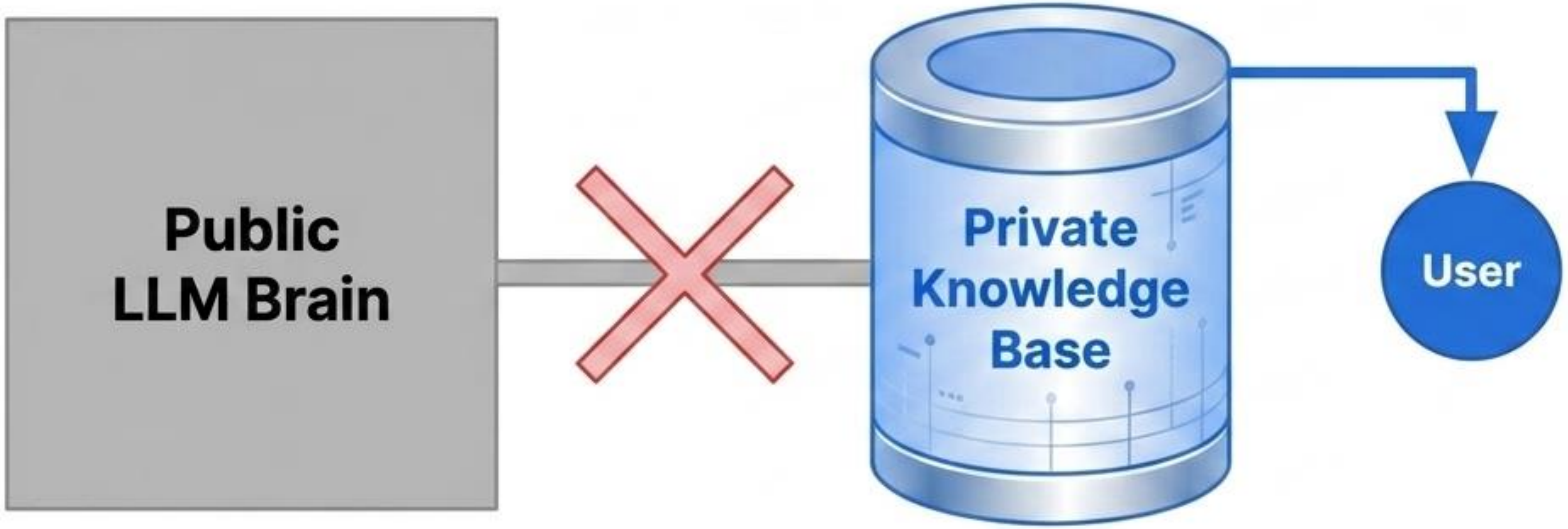
The Friction

Traditional LLMs are trained on millions of generic documents, completely lacking the highly curated, specific context of a targeted video.

The Goal

Transform passive video consumption into an interactive, hyper-specific, and fully searchable knowledge base.

We don’t train a new model. We build a private knowledge base.



The Approach: `Retrieval-Augmented Generation` (RAG).

Why RAG?: Retraining an AI model is extremely costly and inefficient. Instead, **ResuMazing** creates an **isolated, private knowledge base** for a **single video**.

The Result: The model answers questions using only the specific content found within that video, ensuring high relevance and zero generic fluff.

The Ingestion Pipeline: From URL to raw data

Input

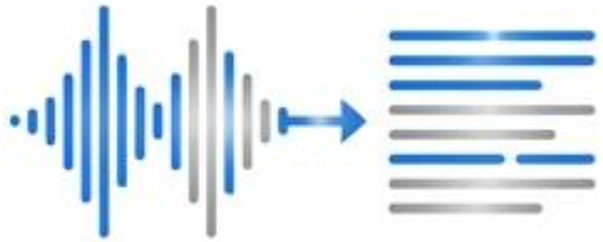
YouTube URL

Audio Extraction

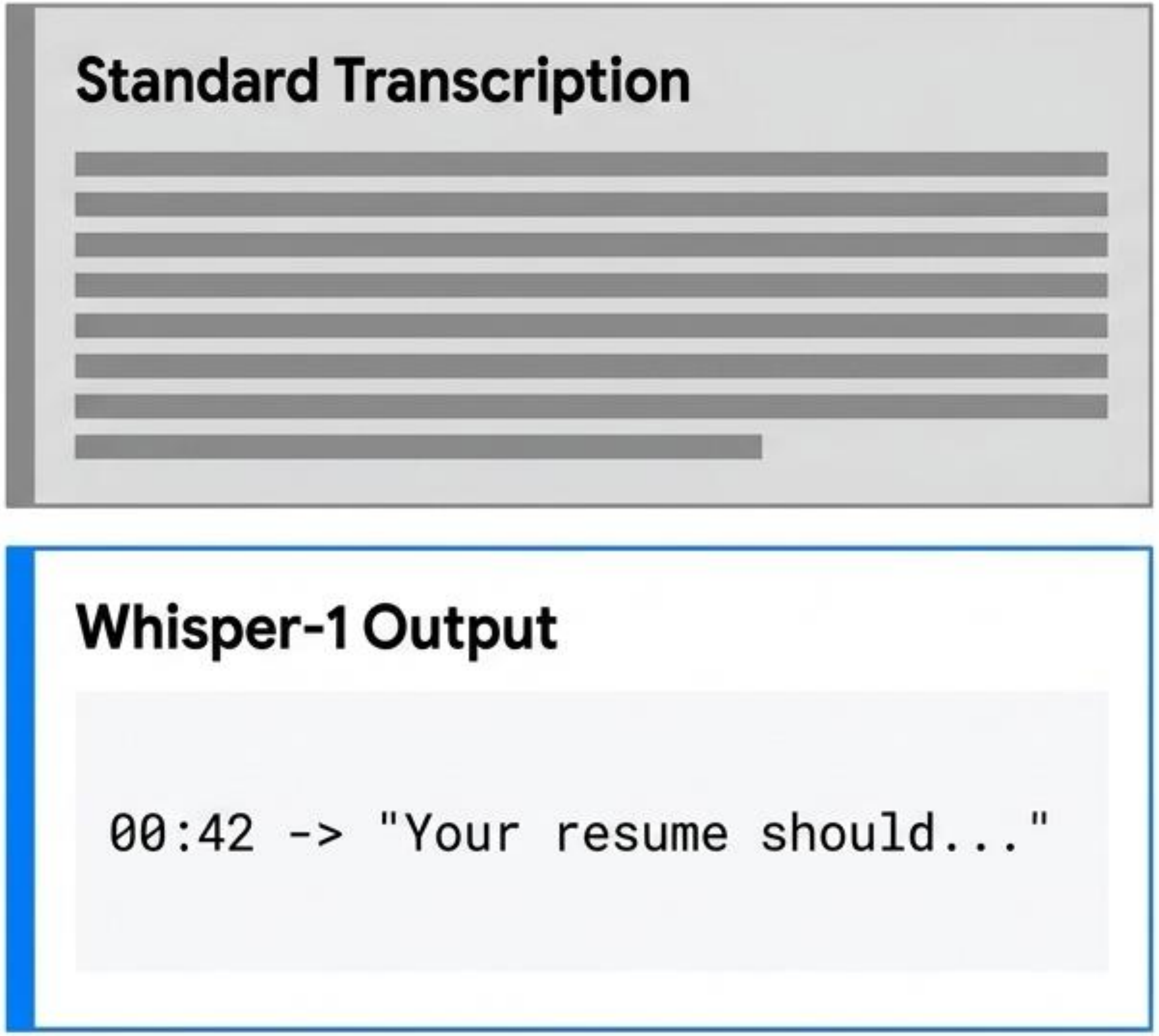
`yt-dlp`

The video track is discarded; only the spoken audio is downloaded locally.

Transcription

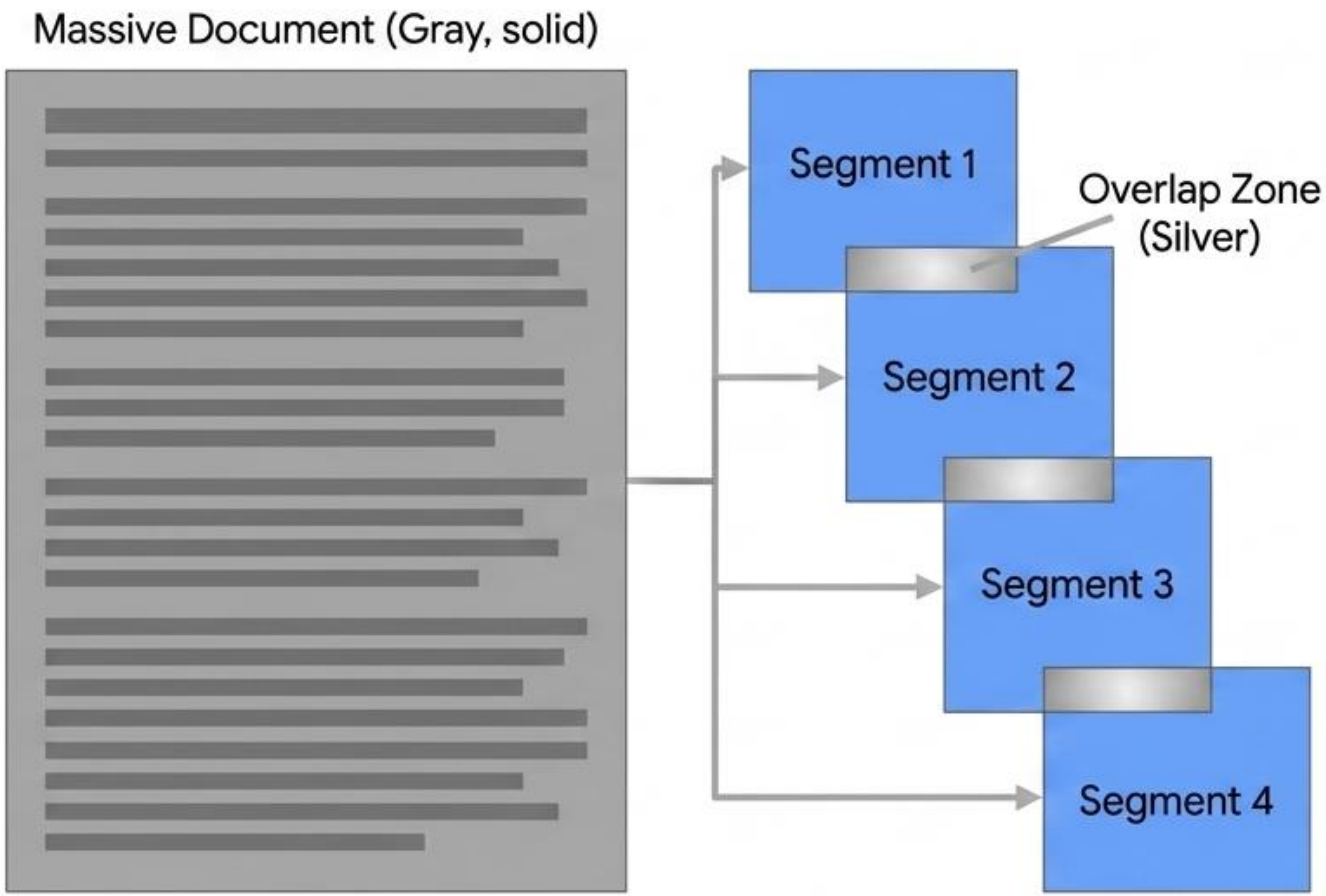


Whisper-1 provides transcription with temporal context



- We utilize OpenAI's Whisper-1 model to process the extracted audio.
- It does not just return a flat transcript. It returns text precisely anchored to a start and end minute.
- Business Value: This allows the final chatbot to provide exact video citations, answering with context like, "The video discusses this at approximately minute 42."

Semantic Chunking protects the context window



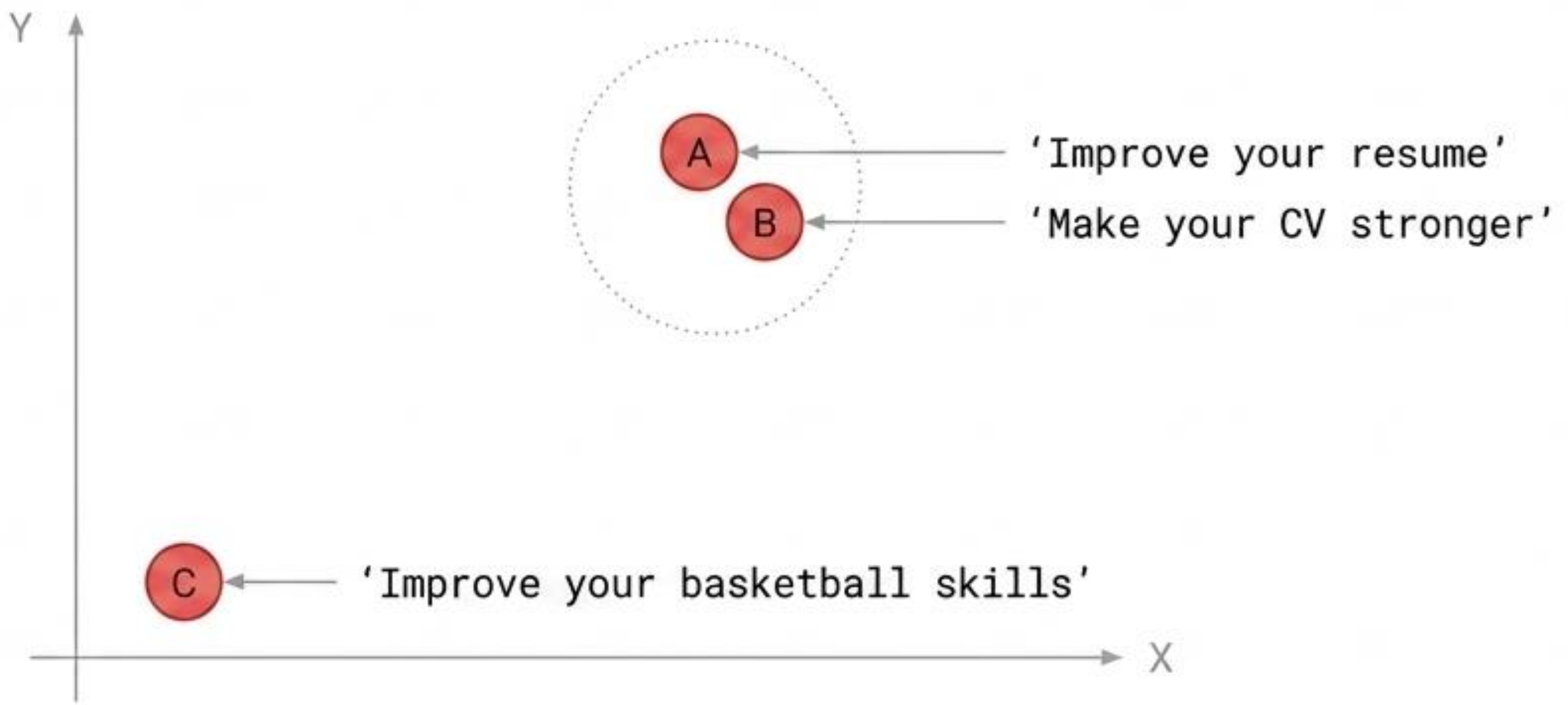
The Constraint: Sending an entire hour-long transcript to a model is cost-prohibitive and exceeds the `LLM context limit`.

The Solution: Divide the content into small, manageable blocks.

The Rule of Overlap: Each block maintains a complete idea but intentionally overlaps with the next segment. This prevents losing critical meaning at the exact point where a cut occurs.

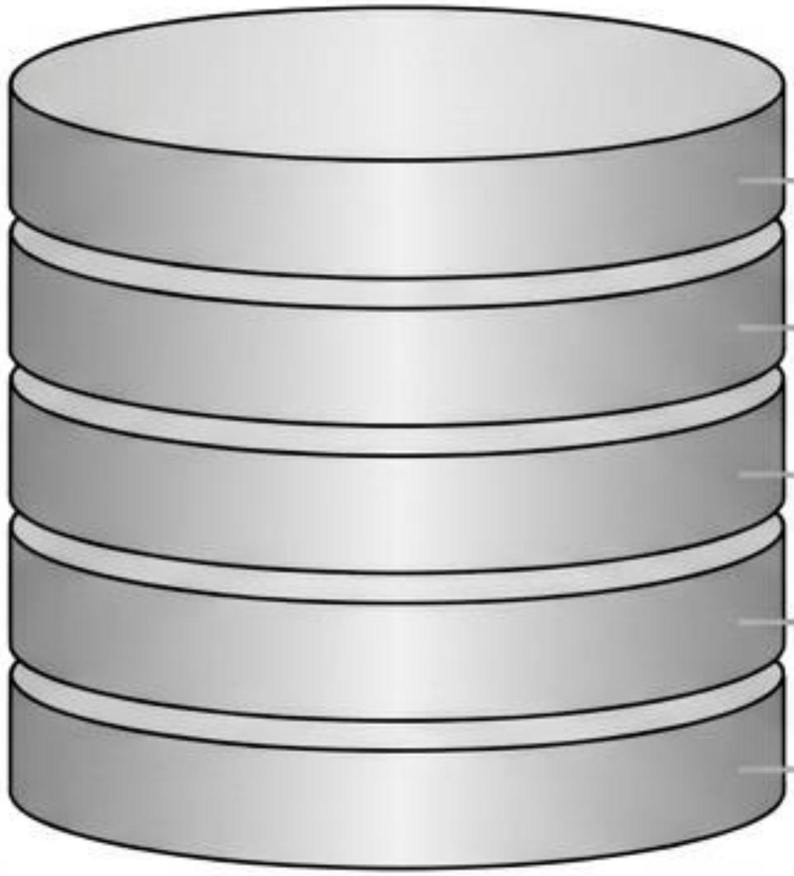
Outcome: One massive document becomes hundreds of small, `vectorized` documents.

Embeddings search for mathematical meaning, not just words



Key Takeaway: Though the words are entirely different, their embeddings place them nearly adjacent in the vector space. The system finds related information even when a user doesn't use the exact vocabulary spoken in the video.

Pinecone: The infrastructure for Vector Similarity Search



Database Breakdown Structure

1. **The Embedding:** `The mathematical vector.`
2. **Original Text:** The human-readable chunk.
3. **Timestamp:** The temporal anchor (`e.g., 00:42`).
4. **Video ID:** The source origin.
5. **Metadata:** Additional contextual tags.

Action: When a query occurs, Pinecone performs a Semantic Search to find vectors mathematically closest to the question's embedding.

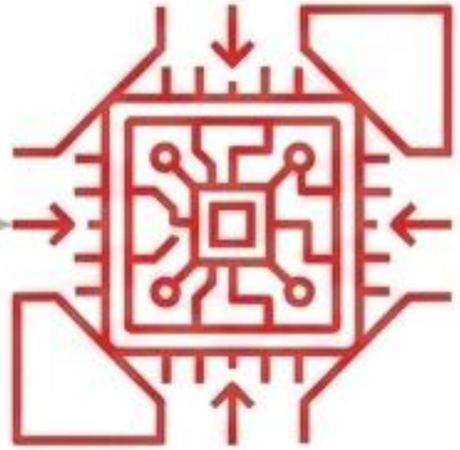
RAG Logic: Separating retrieval from generation

Retrieval (Semantic Retriever)

- Finds information first.
- Searches embeddings within Pinecone to recover the most relevant video fragments.

Generation (GPT-4o-mini)

- Synthesizes the final answer.
- Does not rely on its native knowledge. It generates based solely on a combined payload:
  - The User's Question
  - Retrieved Video Fragments
  - Conversation History
  - Summarized Memory



Query Rewriting ensures precision in the vector search

What about work experience?

(Vague, lacks context)

Backend query

```
What does the video
recommend about
presenting work
experience on a resume?
```

(Complete, specific, independent)

Impact: The Semantic Retriever receives a highly contextualized query, yielding significantly more precise vector results from Pinecone, especially deep into a multi-turn conversation.

Bounding the LLM: Memory and Hallucination Control

Prior Context Retrieval

The Retriever forces the LLM to look at video evidence before formulating any response.

Strict Prompt Engineering

Explicit instructions require the model to answer only with retrieved info, and to explicitly admit when the video lacks the answer.

Temporal Anchoring

Including timestamps in the response forces the model to stay grounded in the original source material.

Memory Compression

Instead of sending the full history, older context is automatically summarized (e.g., 'User wants Data Analyst role, prefers English') to save tokens while maintaining flow.

The Star Feature: The Tool-Calling Agent

What it is:

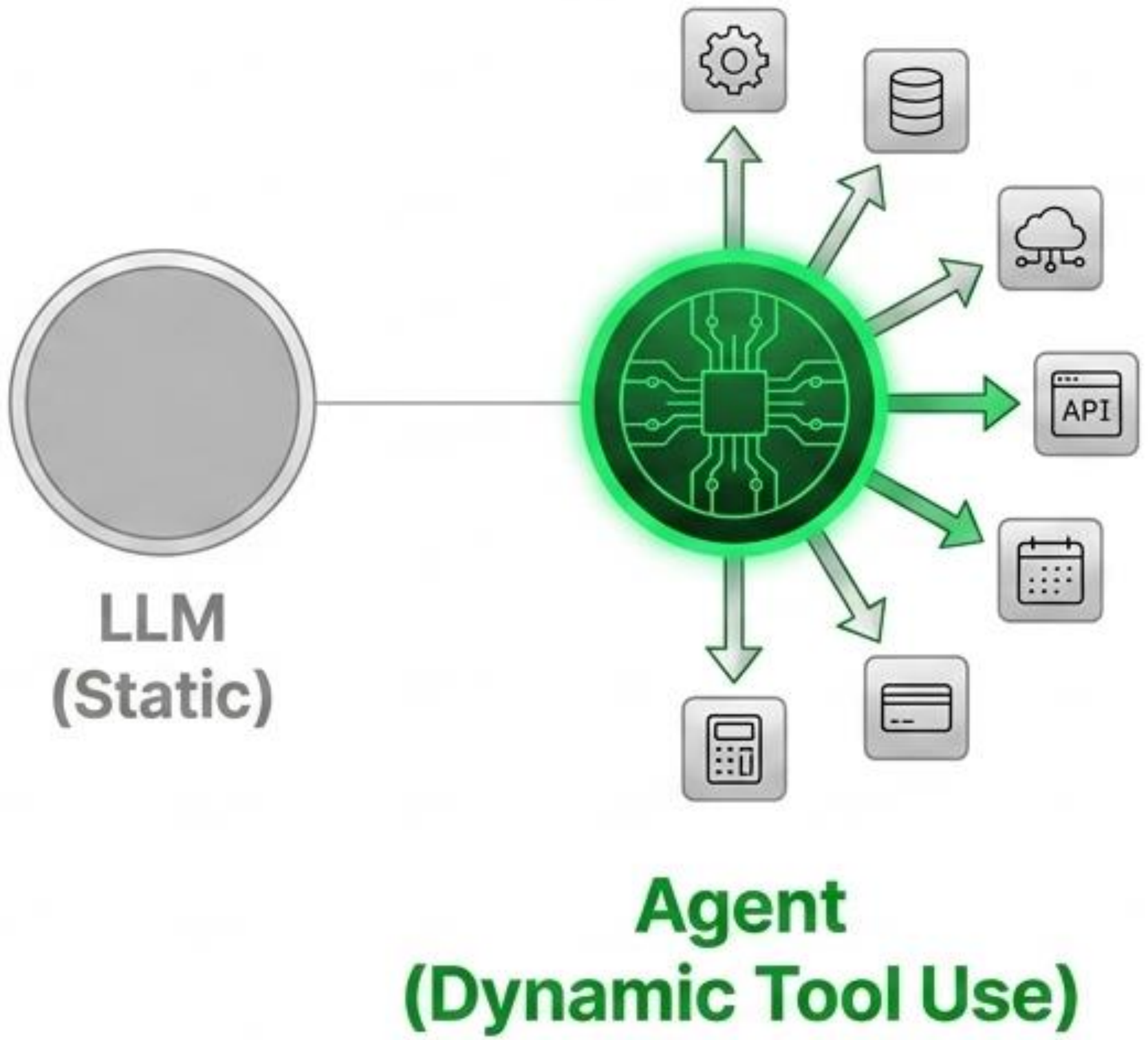
An AI system built with LangChain that extends a Large Language Model's native capabilities by allowing it to interact with external tools and APIs.

What it is NOT:

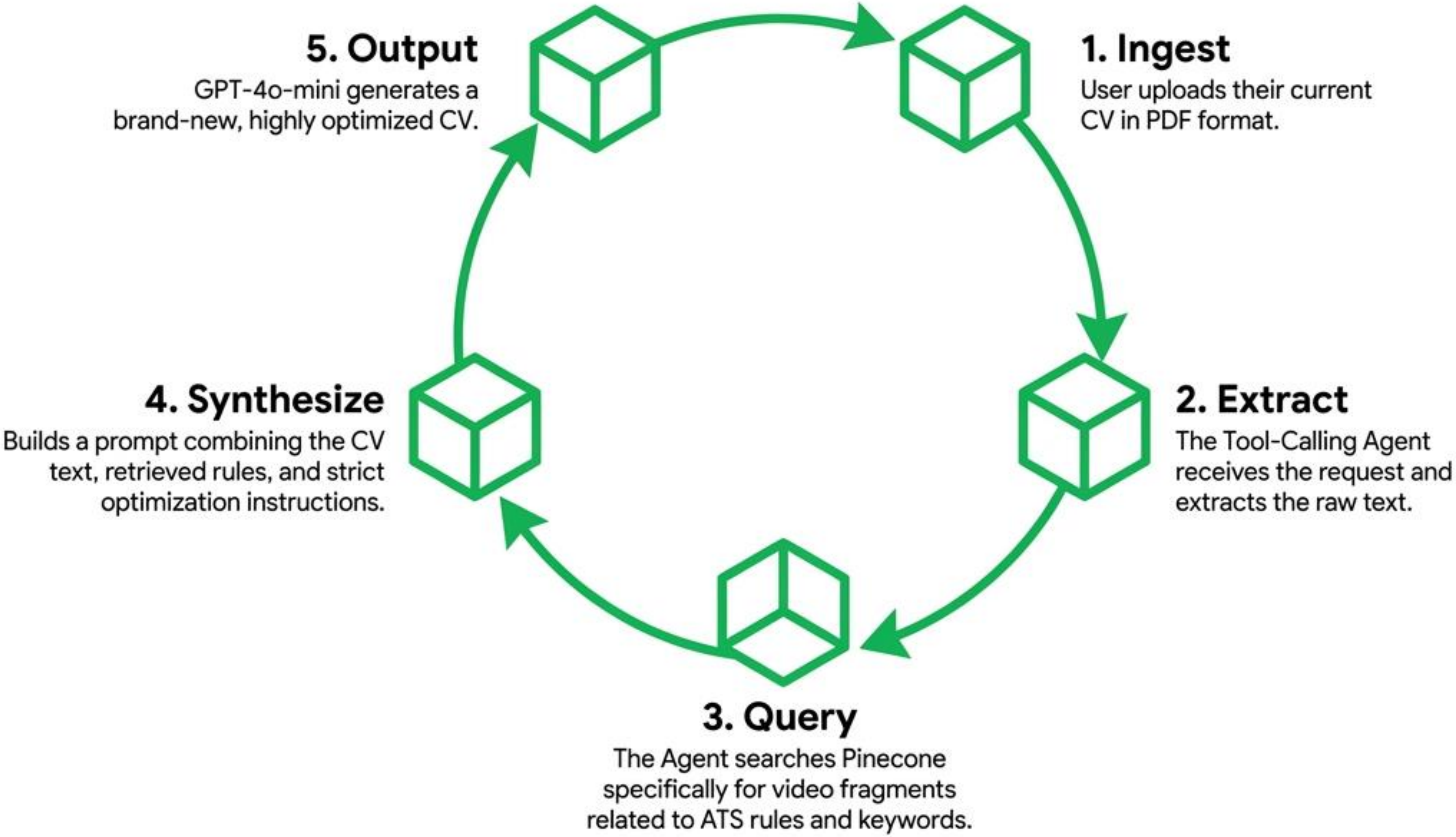
It is not a secondary, separate AI model. It is still GPT-4o-mini.

The Paradigm Shift:

Instead of being limited by static data, the agent autonomously decides when to use tools to fetch real-time data or automate processes. It separates reasoning from business logic.



The PDF CV Analysis Workflow



The Golden Rule of the Optimization Agent

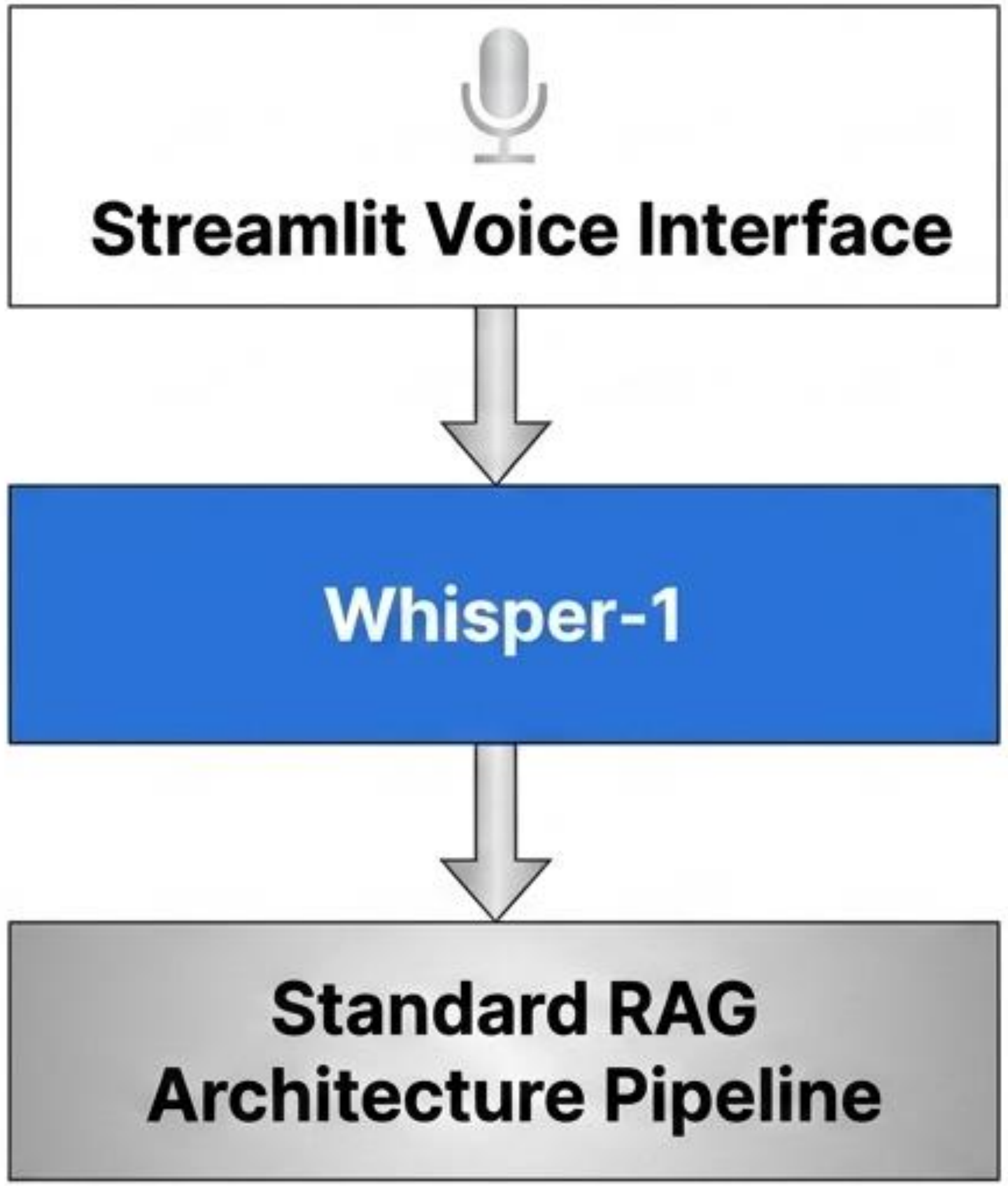
It adapts. It does not invent.

- The Agent is strictly forbidden from inventing professional experience, educational background, or skills.
- Its sole function is to reorganize existing content, improve phrasing, inject relevant keywords, and adapt the structure based purely on the evidence recovered via RAG from the video.

Agent Control & Execution Matrix

LangChain AgentExecutor Parameter	System & Business Value
`max_iterations = 2`	The agent uses the tool in iteration 1 and finalizes the answer in iteration 2. This prevents infinite processing loops and dramatically reduces token consumption.
`verbose = True`	Ensures backend transparency. Allows engineers to visualize every step of the agent's reasoning cycle to verify tool usage during development.
`early_stopping_method = 'force'`	If max iterations are reached without a natural conclusion, execution is forced to stop, guaranteeing a finalized response is always delivered.

A natural user interface built on a robust backend

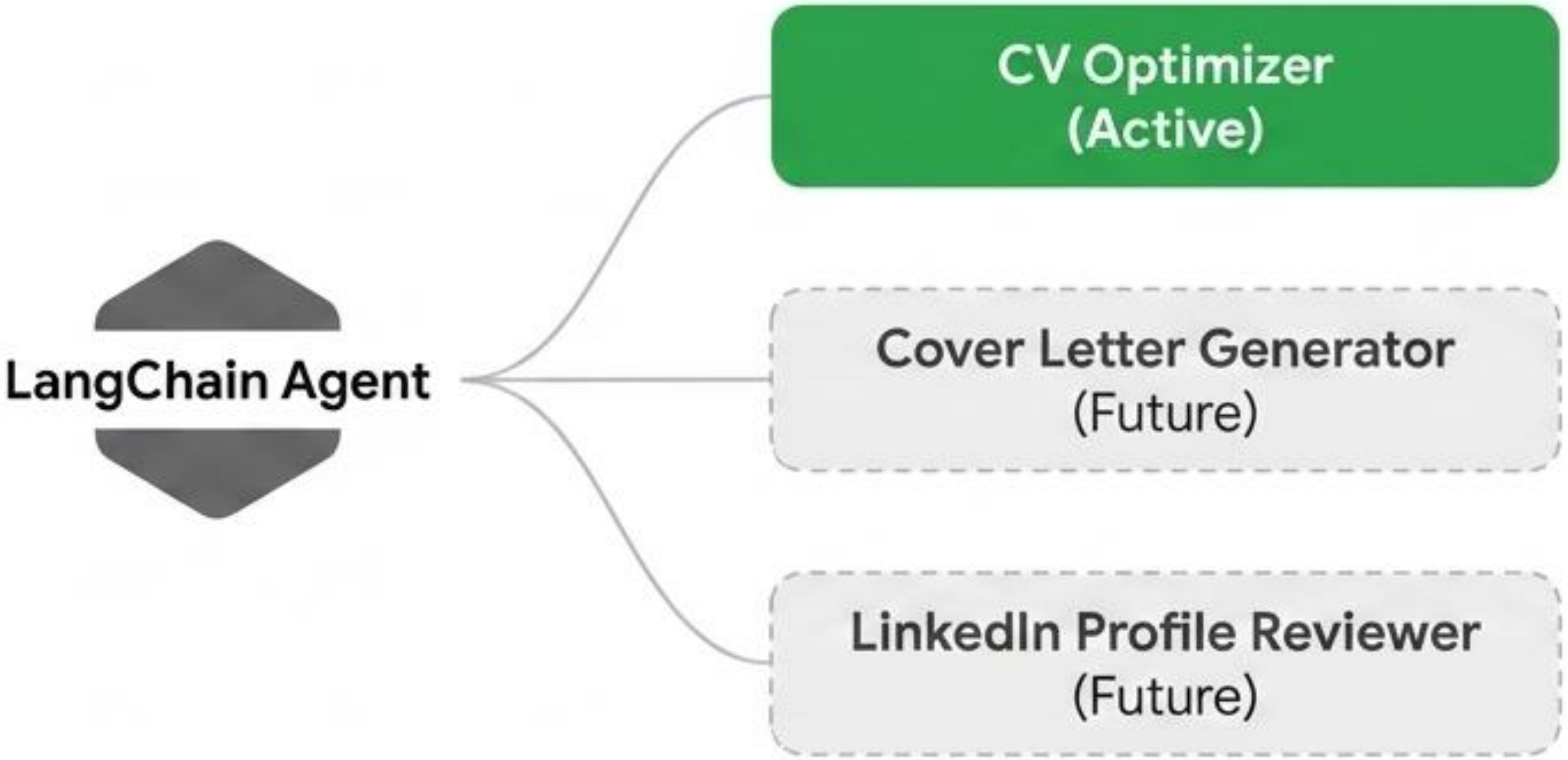


The Interaction:
Users can type or speak natively.

The Routing:
If spoken, Streamlit captures the audio and routes it directly back to Whisper-1 to be converted into text.

The Architecture:
Voice interaction does not change the RAG workflow. The text seamlessly becomes an embedding, triggers a Pinecone retrieval, and routes to GPT-4o-mini. Voice simply adds a frictionless entry point.

Built for scale: Expanding the Agent's toolkit



Because reasoning is separated from business logic, the underlying architecture requires no modification to scale. The same agent will simply decide which tool to pull from its expanded toolkit based on the user's prompt.